

DAOS DFS / DFS 우회 GPU I/O 통합 및 검증 자료

작성 기준: 2026-09-16. 작업 디렉터리: /root/discos_minji.

현재 실행 설정은 두 모드 모두 원래 GPU 백엔드의 /opt/daos-gds-gpu와 /opt/ofi-cuda/lib64를 사용한다. 오브젝트 shim도 이 DAOS 설치본으로 재빌드했다. 아래 3절의 환경 표와 5~7절의 기존 측정값은 전환 전 기록이며, 공통 라이브러리 전환 후 검증은 11절에 따로 정리했다.

이 문서는 두 가지 DAOS 접근 방식을 선택하는 통합 코드, 빌드 절차, 실제 GPU 데이터 왕복 테스트와 DiscoveryBench 축약 벤치마크 결과를 정리한다. /root/discos 원본을 유지하고 discos_minji에서 통합 작업을 수행했다.

현재 두 방식 모두 64MiB GPU 데이터의 저장·읽기·전체 바이트 일치 검증을 통과했다. 오브젝트 모드는 추가로 Qwen3-14B 추론에서 KV 저장과 프로세스 재시작 후 재사용을 확인했다. DFS 모드의 동일 추론 벤치마크는 아직 실행하지 않았다.

1. 통합 목적과 구조

LMCache의 GPU staging과 프리페치 경로를 공통으로 사용하면서, 실제 저장 계층을 DFS 또는 DAOS 오브젝트 API로 선택한다. 기본 코드는 lmcache-daos-repo의 GPU 백엔드이며, discos의 DFS 우회 C shim과 오브젝트 키 매핑을 통합했다.

구분	DFS 사용	DFS 우회
실행 선택	DAOSGDS_TRANSPORT=dfs	DAOSGDS_TRANSPORT=object
LMCache 플러그인	lmcache_daos.gds_backend.DaosGdsBackend	동일
Python 바인딩	dfs_binding.py	object_binding.py
C 라이브러리	libdfs.so → DAOS	libdaosgdr.so → DAOS
GPU 쓰기	dfs_write_gpu	daos_obj_update_gpu
GPU 읽기	dfs_read_gpu	daos_obj_fetch_gpu
키 저장 방식	키 해시를 파일 이름으로 사용	네임스페이스 + LMCache 키 문자열을 dkey로 사용
KV 메타데이터	파일 앞 4KiB 헤더의 JSON	meta akey의 JSON
KV 데이터	파일의 4096바이트 오프셋부터	kv akey
테스트 대상	discospool/kvcache	동일
전용 영역	DFS 디렉터리 /minji-v2	dkey 접두사 minji-v2:

읽기 경로는 다음과 같다. 쓰기는 반대 방향으로 진행된다.



vLLM paged KV cache



모델 추론

오브젝트 모드는 DAOS_OT_MULTI_HASHED, OC_SX 클래스의 고정 오브젝트를 사용한다. OID 생성 입력은 hi=0, lo=1000이며, 클래스 타입 비트는 daos_obj_generate_oid가 반영한다. 각 KV 항목은 서로 다른 dkey 아래에 저장된다. DFS와 오브젝트 모드는 저장 형식과 이름 공간이 달라 서로의 캐시를 자동으로 읽지 않는다.

GPU 직접 I/O는 KV payload의 DAOS 전송 대상이 GPU 메모리라는 의미다. 키·메타데이터·제어 정보에는 호스트 메모리를 사용한다. local_cpu: false여도 LMCache 저장 경로에서 호스트 객체나 CPU allocator가 사용될 수 있으므로, 전체 실행에서 DRAM을 전혀 사용하지 않는다는 뜻은 아니다.

2. 프리페치와 공통 처리

enable_async_loading: true이면 LMCache가 lookup 시점에 존재하는 캐시 청크를 찾고 비동기 읽기를 요청한다. 통합 백엔드는 GPU staging을 할당하고 선택한 DAOS API로 데이터를 가져온다. 읽기가 끝나면 스케줄러에 로드 가능한 토큰 수를 알리고, retrieve 단계에서 staging의 KV를 vLLM KV cache로 옮긴다.

두 모드는 GPU allocator, 읽기·쓰기 작업 풀, 메타데이터 조회 풀, 비동기 프리페치 구현을 공유한다. 현재 설정은 GPU staging 8GiB, I/O 작업 스레드 8개, 메타데이터 작업 스레드 8개다.

_install_multi_prefetch_serializer()는 LMCache의 실행 중 클래스 참조를 바꿔 AsyncMultiSerializer를 사용하게 한다. GPU staging의 청크 예산을 기준으로 여러 요청의 프리페치를 허용하며 DAOS_GDS_MULTI_PREFETCH=0으로 이 교체를 끌 수 있다. GPU로 읽는 동작 자체는 백엔드의 읽기 구현이 담당한다.

현재 serializer의 예산은 프리페치 coroutine 실행 구간에 적용되고, 읽기 완료 후 retrieve까지 staging을 보유하는 전체 시간은 포괄하지 않는다. 다중 요청·장시간 부하에서의 메모리 동작은 별도 검증이 필요하다. 아래 축약 벤치마크는 요청을 순차 실행하므로 요청 간 프리페치 중첩 성능을 측정하지 않는다.

3. 환경 및 설정

다음은 기존 테스트 시 확인한 환경이다. 서버 상태를 재조회한 최신 인벤토리가 아니다.

항목	값
GPU	NVIDIA H100 NVL, nvidia-smi 표시 총 메모리 95830MiB
드라이버 / CUDA 표시	610.57.04 / 13.3
vLLM / LMCache	0.25.1 / 0.5.2
PyTorch	2.11.0+cu130
DFS 클라이언트	/opt/daos-gds-gpu
오브젝트 클라이언트	/opt/discos-daos-gdr
DFS libfabric	/opt/ofi-cuda/lib64
오브젝트 libfabric	/opt/discos-daos-gdr/prereq/release/ofi/lib64
공통 풀 / 컨테이너	discospool / kvcache

항목	값
컨테이너 속성 확인 결과	POSIX, HEALTHY, rd_fac=0

kvcache는 DFS 마운트에 필요한 POSIX 컨테이너이며 오브젝트 API로도 접근할 수 있다. 기존 gdrcont는 layout_type=unknown으로 확인되어 공통 DFS 테스트 대상으로 사용하지 않았다. 복제 없는 OC_SX 오브젝트를 사용하므로 컨테이너의 장애 허용 설정과도 호환되어야 한다.

실제 설정: [lmcache_config_daosgds_unified.yaml](#).

```
chunk_size: 2048
local_cpu: false
enable_async_loading: true
storage_plugins: [daosgds]
extra_config:
  storage_plugin.daosgds.module_path: lmcache_daos.gds_backend
  storage_plugin.daosgds.class_name: DaosGdsBackend
  daosgds.pool: discospool
  daosgds.container: kvcache
  daosgds.transport: dfs
  daosgds.root: /minji-v2
  daosgds.object_namespace: "minji-v2:"
  daosgds.object_library: /root/discos_minji/libdaosgdr.so
  daosgds.gpu_buffer_gb: 8
  daosgds.io_workers: 8
  daosgds.meta_workers: 8
  daosgds.store: true
```

run_vllm.sh는 두 모드에 공통 DAOS·Mercury·libfabric 경로를 설정하고 D_MEM_DEVICE=1, D_GPU_DIRECT=1, PYTHONHASHSEED=0을 전달한다. DAOSGDS_TRANSPORT는 사용할 저장 API를 선택한다. 이 스크립트는 환경변수 미지정 시 dfs를 내보내므로, 스크립트를 사용할 때는 YAML 수정만으로 object 모드가 선택되지 않는다. 아래 명령처럼 환경변수를 명시한다. 모드 변경 시 프로세스를 재시작한다.

4. 빌드와 로딩 확인

오브젝트 모드에서는 로컬 C shim을 빌드한다. DAOS 클라이언트 전체나 서버를 재빌드하는 명령은 아니다.

```
cd /root/discos_minji
make -B all
make check
nm -D ./libdaosgdr.so | rg 'daosgdr_(put|get)(_device)?$'
```

결과물은 /root/discos_minji/libdaosgdr.so다. daosgdr_put_device/get_device가 CUDA 장치 번호를 전달하며 기존 daosgdr_put/get은 장치 0 호환 함수를 유지한다.

DFS 모드는 설치된 GPU 지원 libdfs.so를 사용한다. 이번 작업에서 이 라이브러리를 소스 재컴파일하지 않았으며 Python 컴파일과 동적 로딩·심볼을 검증했다.

```
cd /root/discos_minji
env DAOSGDS_TRANSPORT=dfs ./run_vllm.sh ./venv/bin/python3 -m py_compile \
  lmcache_daos/gds_backend.py lmcache_daos/dfs_binding.py lmcache_daos/serde_v2.py
env DAOSGDS_TRANSPORT=dfs ./run_vllm.sh ./venv/bin/python3 -c \
  'from lmcache_daos.dfs_binding import DfsSys; assert DfsSys.gpu_supported(); print("DFS GPU API available")'
```

이전 단위 테스트 결과는 17 passed였다. 이번 자료 작성에서 테스트를 다시 실행하지는 않았다.

```
cd /root/discos_minji
PYTHONPATH=/root/discos_minji ./venv/bin/python3 -m pytest -q tests
```

5. 64MiB 실제 GPU 왕복 테스트

2026-09-16 재빌드·로딩 검증 이후 실행한 테스트다. GPU에 임의의 uint8 데이터 64MiB를 만들고 저장한 뒤, 별도 GPU 버퍼로 읽어 torch.equal로 전체 바이트를 비교했다. 두 테스트 모두 같은 프로세스 안에서 저장과 읽기를 수행한다.

모드	쓰기 시간	쓰기 GB/s	읽기 시간	읽기 GB/s	결과
object	33.606ms	1.997	55.301ms	1.214	payload·메타데이터 일치, 키 삭제 후 부재 확인
dfs	25.093ms	2.674	5.487ms	12.231	payload 일치, 임시 파일 삭제 성공

64MiB는 67,108,864바이트이며 대역폭의 GB는 10억 바이트 기준이다. 결과는 당시 도구 실행 출력에서 옮겼다. 해당 64MiB 실행의 별도 로그 파일은 저장하지 않았으며, 아래 명령으로 새로운 로그를 남길 수 있다.

```
cd /root/discos_minji
set -o pipefail
env DAOSGDS_TRANSPORT=object ./run_vllm.sh ./venv/bin/python3 \
  tests/object_gpu_roundtrip.py --pool discospool --container kvcache --size-mib 64 \
  2>&1 | tee "object_gpu_roundtrip_$(date +%Y%m%d_%H%M%S).log"

env DAOSGDS_TRANSPORT=dfs ./run_vllm.sh ./venv/bin/python3 \
  tests/dfs_gpu_roundtrip.py --pool discospool --container kvcache \
  --root /minji-v2 --size-mib 64 \
  2>&1 | tee "dfs_gpu_roundtrip_$(date +%Y%m%d_%H%M%S).log"
```

두 스크립트는 UUID가 포함된 테스트 키·파일을 사용한다. 성공한 실행에서 해당 데이터만 삭제했다. DFS 테스트가 만든 /minji-v2 디렉터리는 유지된다.

측정 구간은 각 데이터 쓰기·읽기 호출과 CUDA 동기화다. 버퍼 생성, 바이트 비교, 삭제는 제외한다. DFS 테스트의 파일 열기·닫기도 시간에서 제외된다. DFS 테스트는 오프셋 0의 원시 payload를 검증하므로 통합 백엔드의 4KiB 헤더·임시 파일 rename 경로 전체를 검증한 것은 아니다.

이 수치는 단발성 기능 테스트 결과다. 두 방식은 DAOS·libfabric 클라이언트 빌드, 저장 매핑, 요청 분할이 다르고 워밍업·반복 측정도 통제하지 않았다. 따라서 읽기 시간 차이를 DFS 계층 유무만의 효과로 해석하거나 성능 우열로 확정할 수 없다.

6. 오브젝트 모드 DiscoveryBench 축약 벤치마크

2026-09-14에 [kv_measure.py](#)로 실행했다. 실제 Qwen3-14B 추론을 수행하지만 생성한 분석 코드를 실행하지 않고 출력 앞 120자를 다음 턴 히스토리에 넣는 축약 워크로드다. full agentic 실행·과학적 정답 채점은 포함하지 않는다.

조건	값
모델	Qwen/Qwen3-14B, BF16
태스크 / 턴	adventure travel, 5컬럼, 1개 태스크 × 6턴
모드	object, 비동기 로딩 활성화
청크	2048토큰, KV payload 320MiB
최대 모델 길이 / 생성 길이	8192 / 기본 64토큰
고정 prefix padding	--pad-tokens 3000 — 근사치, 실제 첫 프롬프트 2679토큰
GPU staging / I/O workers	8GiB / 8
vLLM 설정	내장 prefix caching 비활성화, enforce_eager=True, 메모리 사용 비율 0.75
샘플링	temperature=0

토큰당 KV 크기는 40층 × K/V 2 × 8 heads × 128 dimensions × 2 bytes = 163,840 bytes다. 청크 2048개 토큰을 곱하면 335,544,320바이트, 즉 320MiB가 된다.

턴	프롬프트 토큰	run1 cached	run2 cached	run2 재사용률
1	2679	0	2048	76.4%
2	2705	2048	2048	75.7%
3	2735	2048	2048	74.9%
4	2765	2048	2048	74.1%
5	2795	2048	2048	73.3%
6	2825	2048	2048	72.5%

run1에서 한 청크의 GPU 쓰기 호출은 135.100ms였다. run2에서 여섯 GPU 읽기 호출은 66.010~67.616ms였고, 백엔드 prefetch 전체 시간은 약 66.7~68.6ms, 약 4.9~5.0GB/s였다. 단일 청크 순차 요청에서 관측한 값이며 포화 대역폭을 뜻하지 않는다.

run2는 새 프로세스에서 시작했고 첫 요청부터 DAOS의 2048토큰을 읽었다. 서버 캐시를 유지한 채 프로세스 내 인덱스가 사라져도 캐시 재사용이 가능함을 확인했다. 이 벤치마크는 저장 전후 전체 KV 바이트나 생성 토큰의 동일성 비교를 수행하지 않았다. 전체 바이트 검증 근거는 별도의 64MiB 왕복 테스트다.

원본 증거:

- [run1 CSV](#), [run1 로그](#)
- [run2 CSV](#), [run2 로그](#)

LMCache 로그의 Retrieved ... throughput는 이미 프리페치된 데이터를 소비하는 구간일 수 있다. 수백 GB/s로 보이는 이 값을 DAOS 네트워크 읽기 대역폭으로 사용하면 안 된다. DAOS I/O 시간은 daos_obj_fetch_gpu 또는 DaosGdsBackend prefetch 로그로 확인한다.

7. 벤치마크 중 발견한 이벤트 정리 문제

run1에서는 캐시 hit 후 MemoryObj 참조 카운트가 음수가 되는 이중 해제 경고가 5회 발생했다. 설치된 LMCache 코드에서 retrieve가 객체를 해제한 뒤에도 완료된 프리페치 이벤트가 남아, 정상 요청의 lookup_unpin이 같은 객체를 다시 정리하는 경로를 확인했다.

통합 백엔드에 _install_consumed_prefetch_event_patch()를 추가해 _async_process_tokens_internal이 반환된 후 완료 이벤트를 제거하도록 했다. 프리페치만 완료되고 아직 소비되지 않은 요청은 기존 cleanup 경로에 남기는 의도다. 패치는 실행 중 LMCache 클래스에 적용되며 discos_minji 밖의 설치 파일은 수정하지 않는다.

run2에서 해당 경고가 재현되지 않았고 당시 단위 테스트 17개가 통과했다. 다만 이 패치의 취소·예외·다중 요청 경합 전체가 검증된 것은 아니다. LMCache 내부 메시드에 의존하므로 버전 변경 시 재검토해야 한다. run1과 run2 사이 코드가 달라졌으므로 두 실행의 시간을 동일 코드의 엄밀한 cold/warm 성능 비교로 사용하지 않는다.

8. 벤치마크 재실행

다음은 기존 조건으로 오브젝트 모드를 두 프로세스에서 순서대로 실행하고 새 디렉터리에 결과를 저장하는 명령이다.

```
cd /root/discos_minji
set -o pipefail
bench_dir=$(mktemp -d /root/discos_minji/object_bench.XXXXXX)
for run in run1 run2; do
  env DAOSGDS_TRANSPORT=object LMCACHE_LOG_LEVEL=DEBUG \
  ./run_vllm.sh ./venv/bin/python3 ./kv_measure.py \
  --root /root/discos_minji/discoverybench/discoverybench/synth/train \
  --model Qwen/Qwen3-14B --max-model-len 8192 --tasks 1 --steps 6 \
  --chunk-size 2048 --pad-tokens 3000 --no-prefix-cache \
  --out "$bench_dir/$run.csv" 2>&1 | tee "$bench_dir/$run.log" || break
done
```

기존 minji-v2: 캐시가 남아 있으므로 재실행의 run1도 첫 요청부터 hit할 수 있다. 새 cold/warm 실험은 YAML의 daosgds.object_namespace를 실험별 새 접두사로 바꾸고 두 실행에서 동일하게 유지한다. 공용 컨테이너 전체를 삭제할 필요가 없다. DFS 비교에서는 새 daosgds.root를 지정하고 DAOSGDS_TRANSPORT=dfs로 실행한다. 해당 DFS 추론 실험은 후속 검증 절차이며 이 문서의 완료 결과에 포함하지 않는다.

복사된 venv/bin/vllm 등의 실행 파일에는 /root/discos/venv를 가리키는 shebang이 남아 있다. 위 명령은 discos_minji/venv/bin/python3를 직접 사용한다. 기존 sweep_q14.sh에는 원본 /root/discos 설정 수정과 gdrcont 삭제 코드가 남아 있으므로, 통합 비교 실험에는 위 명령을 사용한다.

9. 검증 범위와 후속 비교 조건

항목	object	dfs
라이브러리 준비	C shim 재빌드·링크 확인	설치된 libdfs 로딩·GPU 심볼 확인

항목	object	dfs
DAOS 서버 연결	성공	성공, 루트 stat 확인
64MiB GPU 데이터 전체 바이트 비교	통과	통과
통합 백엔드 Qwen3-14B 추론	축약 벤치마크 실행	미실행
프로세스 재시작 후 KV 재사용	확인	미검증
통합본 full agentic DiscoveryBench	미실행	미실행
동일 조건 반복 성능 비교·TTFT 비교	미완료	미완료

성능 비교를 이어갈 때는 동일한 통합 코드 버전, 모델, 체크, GPU staging, 요청·프롬프트, 캐시 초기 상태를 유지해야 한다. 클라이언트·libfabric 빌드 차이를 기록하고 가능하면 공통 빌드에서 두 API를 비교한다. DFS warm-up 유무도 통제하고 반복 측정의 중앙값·분산, TTFT, DAOS I/O 시간, KV hit 수를 함께 기록한다.

10. 관련 구현 파일

파일	역할
gds_backend.py	모드 선택, GPU staging, async 조회·읽기, 런타임 패치
dfs_binding.py	DFS 마운트·파일·GPU I/O 바인딩
object_binding.py	C shim 호출, 메타데이터 조회, 오류 처리
libdaosgdr.c	DAOS 오브젝트 직접 GPU I/O와 dkey/akey 구성
serde_v2.py	DFS 키 경로와 헤더 형식
Makefile	오브젝트 C shim 빌드와 링크 검사
run_vllm.sh	모드별 라이브러리·실행 환경 선택
object_gpu_roundtrip.py	실제 오브젝트 GPU 왕복·메타데이터 비교
dfs_gpu_roundtrip.py	실제 DFS GPU 왕복·바이트 비교
INTEGRATED_TRANSPORTS.md	기존 영문 통합 안내

11. 공통 클라이언트 전환 및 재검증

2026-09-16, run_vllm.sh에서 object 모드의 전용 클라이언트 선택을 제거했다. DFS와 object 모두 /opt/daos-gds-gpu를 사용하고 Makefile의 기본 DAOS_PREFIX도 같은 경로로 바꿨다. make -B all로 shim을 재빌드하고 링크 검사를 통과했다. /root/discos와 /opt의 설치 파일은 변경하지 않았다.

설정값뿐 아니라 GPU I/O를 실제 수행한 프로세스의 /proc/self/maps를 검사했다. 두 프로세스에서 다음 파일이 동일하게 로드됐다.

구성 요소	실제 로드 경로
DAOS	/opt/daos-gds-gpu/lib64/libdaos.so.2.8.0
Mercury	/opt/daos-gds-gpu/prereq/release/mercury/lib64/libmercury.so.2.4.1
libfabric	/opt/ofi-cuda/lib64/libfabric.so.1.25.0

추가로 object는 /root/discos_minji/libdaosgdr.so, DFS는 /opt/daos-gds-gpu/lib64/libdfs.so를 로드했다. 위 파일명에 포함된 버전 숫자는 로드된 파일 이름 그대로이며 별도 API 버전 조회 결과를 뜻하지 않는다.

공통 라이브러리로 각각 64MiB 테스트를 한 번 실행한 결과:

모드	쓰기 시간	쓰기 GB/s	읽기 시간	읽기 GB/s	검증
object	29.846ms	2.249	15.365ms	4.368	전체 payload·메타데이터 일치, 테스트 키 삭제
dfs	21.175ms	3.169	3.623ms	18.522	전체 payload 일치, 테스트 파일 삭제

이 결과로 공통 설치본에서도 두 GPU I/O 경로가 동작함을 확인했다. 한 번씩 실행했고 저장 구조·워킹업·측정 구간 차이가 남아 있어 성능 우열이나 기존 측정 대비 개선 폭을 확정하지 않는다. 이번 전환 후 vLLM 벤치마크는 아직 재실행하지 않았다.

로그: [object 공통 라이브러리 검증](#), [DFS 공통 라이브러리 검증](#).

GPU 왕복과 실제 라이브러리 경로를 함께 검사하는 재실행 명령:

```
cd /root/discos_minji
DAOSGDS_TRANSPORT=object ./run_vllm.sh ./venv/bin/python3 tests/check_common_gpu_stack.py
DAOSGDS_TRANSPORT=dfs ./run_vllm.sh ./venv/bin/python3 tests/check_common_gpu_stack.py
```

성공 시 payload=match, LOADED ..., PASS common native stack을 출력한다. [check_common_gpu_stack.py](#)는 기존 왕복 테스트를 호출한 후 로드된 공통 라이브러리 경로를 검증한다.